

Лабораторная работа №7

Элементы криптографии. Однократное гаммирование

Сергеев Даниил Олегович

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
3	Ответы на контрольные вопросы	15
4	Выводы	17
	Список литературы	18

Список иллюстраций

2.1	Вывод программы без аргументов	12
2.2	Шифрование фразы Stirlitz - You are a Hero!!	12
2.3	Ключ для Happy New Year, Friends!!!! и шифротекста . .	13
2.4	Расшифровка фразы из нового ключа и старого шифротекста	14
2.5	Создание ключа для фразы Stirlitz - You are a Fool!! .	14
2.6	Расшифровка фразы Stirlitz - You are a Fool!!	14

Список таблиц

1 Цель работы

Освоить на практике применение режима однократного гаммирования. [1]

2 Выполнение лабораторной работы

Войдем в систему и создадим файл программы на языке C в каталоге `~/testing`. Назовем его `gamma.c`.

```
cd ~/testing  
vi gamma.c
```

Будем принимать ключ, открытый текст и шифротекст через аргументы программы:

- Опция `-k` : ключ в hex формате
- Опция `-o` : открытый текст в hex формате
- Опция `-e` : шифротекст в hex формате
- Формат: `00 01 02 ... 1F 2F ...`

Для удобства будем использовать байт-буферы фиксированного размера. Сложение по модулю будем применять для всех элементов двух массивов.

На выход программа будет выводить введенные данные (открытый текст будет отображен в символьном и в шестнадцатеричном формате). В зависимости от входных аргументов:

- Если есть ключ и открытый текст (`new_text`, `test_decrypt`): будет выводиться шифротекст, полученный путем применения сложения по модулю ключа и текста;

- Если есть шифротекст и ключ (decrypt): будет выводиться расшифровка шифротекста (в байтах и в строчном виде);
- Если есть шифротекст и открытый текст (desired_key): будет выводиться ключ, необходимый для преобразования шифротекста в приведенный открытый текст.

Листинг №1. Код программы

```
#include <stdio.h>
#include <string.h>
#include <locale.h>
#include <stdbool.h>

#define MAX_BUFFER_SIZE 40
typedef unsigned char byte;

void char_2_hex(const char c, byte *val)
{
    if ((c >= '0') && (c <= '9'))
        *val = (byte)(c - '0');
    else if ((c >= 'a') && (c <= 'f'))
        *val = (byte)(c - 'a') + 10;
    else if ((c >= 'A') && (c <= 'F'))
        *val = (byte)(c - 'A') + 10;
}

void read_bytes(byte *buffer, char *data)
{

```

```

    int i = 0, j = 0;
    while (data[i] != '\0')
    {
        byte left = 0;
        byte right = 0;

        char_2_hex(data[i], &left);
        char_2_hex(data[i + 1], &right);

        buffer[j] = (left << 4) | (right & 0x0f);
        if (data[i + 2] == '\0') break;
        i += 3; j++;
    }
}

void print_as_bytes(byte *buffer)
{
    for (int i = 0; i < MAX_BUFFER_SIZE; i++)
    {
        printf("%02X ", buffer[i]);
    }
    printf("\n");
}

void print_as_chars(byte *buffer)
{
    for (int i = 0; i < MAX_BUFFER_SIZE; i++)

```



```

    {
        printf("%c", buffer[i]);
    }
    printf("\n");
}

void xor(byte *buffer, byte *open, byte *key) {
    for (int i = 0; i < MAX_BUFFER_SIZE; i++)
    {
        buffer[i] = open[i] ^ key[i];
    }
}

int main(int argc, char **argv)
{
    byte      key[MAX_BUFFER_SIZE] = { 0 };
    byte      open[MAX_BUFFER_SIZE] = { 0 };
    byte encrypted[MAX_BUFFER_SIZE] = { 0 };

    bool hasKey = false;
    bool hasOpen = false;
    bool hasEncrypted = false;

    int i = 1;
    while (i < argc)
    {
        if (strcmp(argv[i], "-k") == 0) {
            if (i + 1 < argc)

```

```

        {
            read_bytes(key, argv[i + 1]);
            hasKey = true;
        }
    }
    else if (strcmp(argv[i], "-o") == 0) {
        if (i + 1 < argc)
        {
            read_bytes(open, argv[i + 1]);
            hasOpen = true;
        }
    }
    else if (strcmp(argv[i], "-e") == 0) {
        if (i + 1 < argc)
        {
            read_bytes(encrypted, argv[i + 1]);
            hasEncrypted = true;
        }
    }
    i++;
}

printf("key:      (byte)");
print_as_bytes(key); printf("\n");
printf("open:     (byte)");
print_as_bytes(open); printf("\n");
printf("          (char)");
print_as_chars(open); printf("\n");

```

```

printf("encrypted: (byte)");
print_as_bytes(encrypted); printf("\n");

byte buffer[MAX_BUFFER_SIZE] = { 0 };
if (hasOpen && hasKey) {
    xor(buffer, open, key);
    printf("new_text:      (byte)");
    print_as_bytes(buffer); printf("\n");
    xor(buffer, buffer, key);
    printf("test_decrypt: (byte)");
    print_as_bytes(buffer); printf("\n");
    printf("                (char)");
    print_as_chars(buffer); printf("\n");
}
if (hasEncrypted && hasKey) {
    xor(buffer, encrypted, key);
    printf("decrypt:      (byte)");
    print_as_bytes(buffer); printf("\n");
    printf("                (char)");
    print_as_chars(buffer); printf("\n");
}
if (hasEncrypted && hasOpen) {
    xor(buffer, encrypted, open);
    printf("desired_key:  (byte)");
    print_as_bytes(buffer); printf("\n");
}
}

```

Скомпилируем программу и проведем пробный запуск.

```
cd ~/testing
ls
gcc gamma.c -o gamma
./gamma
```

[illegible]

Рисунок 2.1: Вывод программы без аргументов

Будем использовать латиницу для записи сообщений. Сначала попробуем зашифровать сообщение `Stirlitz - You are a Hero!!` с помощью ключа той же длины.

- Текст: 53 74 69 72 6C 69 74 7A 20 2D 20 59 6F 75 20 61 72 65 20 61 20 48 65 72 6F 21 21
- (``Stirlitz - You are a Hero!!``)
- Ключ: 05 0C 17 7F 0E 4E 37 D2 94 10 09 2E 22 57 FF C8 0B B2 70 54 55 F4 D3 07 BB BC 54
- Результат: 56 78 7E 0D 62 27 43 A8 B4 3D 29 77 4D 22 DF A9 79 D7 50 35 75 BC B6 75 D4 9D 75

```
dosergeev@dosergeev testing$ ./gmm -o "53 74 69 72 6C 69 74 7A 20 2D 20 59 6F 75 20 61 72 65 20 61 20 48 65 72 6F 21 21" -k "05 0C 1F 7F 0
E 4E 3F 02 94 10 0E 22 57 FF 08 08 02 54 55 64 F3 07 BB 8C 54"
key: (byte)95 0C 17 7F 0E 4E 3F 02 94 10 0E 22 57 FF 08 08 02 70 54 55 F3 07 BB 8C 54 00 00 00 00 00 00 00 00 00 00 00 00 00 00
open: (byte)53 74 69 72 6C 69 74 7A 20 2D 20 59 6F 75 20 61 72 65 20 61 20 48 65 72 6F 21 21 00 00 00 00 00 00 00 00 00 00 00
(char)Stirlitz - You are a Hero!!
encrypted: (byte)00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
new_text: (byte)56 78 7E 0D 62 27 43 A8 B4 3D 29 77 4D 22 DF A9 79 D7 50 35 75 8C 86 75 D4 9D 70 50 00 00 00 00 00 00 00 00 00 00
test_decrypt: (byte)53 74 69 72 6C 69 74 7A 20 2D 20 59 6F 75 20 61 72 65 20 61 20 48 65 72 6F 21 21 00 00 00 00 00 00 00 00 00 00 00
(char)Stirlitz - You are a Hero!!
[dosergeev@dosergeev testing$ ]
```

Рисунок 2.2: Шифрование фразы Stirlitz - You are a Hero!!

Проверка ключа показывает, что мы правильно создали шифротекст. Теперь попробуем создать ключ для открытого текста `Happy New Year, Friends!!!!` и зашифрованного текста, полученного ранее.

- Текст: 48 61 70 70 79 20 4E 65 77 20 59 65 61 72 2C 20 46 72 69 65 6E 64 73 21 21 21 21
- `(`Happy New Year, Friends!!!!`)`
- Шифротекст: 56 78 7E 0D 62 27 43 A8 B4 3D 29 77 4D 22 DF A9 79 D7 50 35 75 BC
B6 75 D4 9D 75
- Результат: 1E 19 0E 7D 1B 07 0D CD C3 1D 70 12 2C 50 F3 89 3F A5 39 50 1B D8 C5
54 F5 BC 54

[illegible]

Рисунок 2.3: Ключ для Happy New Year, Friends!!!! и шифротекста

Теперь попробуем расшифровать фразу, используя новый ключ и тот же шифротекст.

- Шифротекст: 56 78 7E 0D 62 27 43 A8 B4 3D 29 77 4D 22 DF A9 79 D7 50 35 75 BC B6 75 D4 9D 75
- Ключ: 1E 19 0E 7D 1B 07 0D CD C3 1D 70 12 2C 50 F3 89 3F A5 39 50 1B D8 C5 54 F5 BC 54
- Результат: 48 61 70 70 79 20 4E 65 77 20 59 65 61 72 2C 20 46 72 69 65 6E 64 73 21 21 21 21
- (`Happy New Year, Friends!!!!`)

```
[dosergeev@dosergeev testing]$ ./gamma -k "1E 19 0E 70 1B 07 00 C3 D3 10 72 2C 50 F3 89 3F A5 39 50 1B D8 C5 54 F5 BC 54" -e "56 78 TE 00
2 27 43 A8 B4 3D 29 77 40 22 DF A9 79 D7 50 35 75 BC 86 75 D4 90 75"
key: (byte)1E 19 0E 70 1B 07 00 C3 D3 10 72 2C 50 F3 89 3F A5 39 50 1B D8 C5 54 F5 BC 54 00 00 00 00 00 00 00 00 00 00 00 00 00 00
open: (byte)00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
(char)
encrypted: (byte)56 78 TE 00 62 27 43 A8 B4 3D 29 77 40 22 DF A9 79 D7 50 35 75 BC 86 75 D4 90 75 00 00 00 00 00 00 00 00 00 00 00 00
decrypt: (byte)48 61 70 70 70 20 4E 65 77 20 59 65 61 72 2C 20 46 72 69 65 6E 64 73 21 21 21 00 00 00 00 00 00 00 00 00 00 00 00
(char)Happy New Year, Friends!!!!

[dosergeev@dosergeev testing]$
```

Рисунок 2.4: Расшифровка фразы из нового ключа и старого шифротекста

Фраза расшифровалась правильно. Дополнительно создадим ключ для фразы Stirlitz - You are a Fool!! из теоретического введения.

[illegible]

Рисунок 2.5: Создание ключа для фразы Stirlitz - You are a Fool!!

```
[dosergeev@dosergeev testing]$ ./gamma -k "05 0c 17 7f 0e 4e 37 02 94 10 09 2e 22 57 ff c8 08 b2 76 54 55 fa d9 1a 8a bc 54" -e "56 78 7e 0d  
2 27 43 ab 84 30 29 77 4d 22 df a9 70 d7 50 35 75 bc 86 75 d4 90 75"  
key: (byte)05 0c 17 7f 0e 4e 37 d2 94 10 09 2e 22 57 ff c8 08 b2 76 54 55 fa d9 1a 8b 8c 54 00 00 00 00 00 00 00 00 00 00 00 00 00 00  
open: (byte)00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  
(char)  
encrypted: (byte)56 78 7e 0d 62 27 43 ab 84 30 29 77 4d 22 df a9 79 d7 50 35 75 bc 86 75 d4 90 75 00 00 00 00 00 00 00 00 00 00 00 00 00  
decrypt: (byte)53 74 69 72 6c 69 74 7a 20 2d 20 59 6f 75 20 61 72 65 20 61 20 46 6f 6f 6c 21 21 00 00 00 00 00 00 00 00 00 00 00 00 00  
(char)Stirlitz - You are a Fool!!  
[dosergeev@dosergeev testing]$
```

Рисунок 2.6: Расшифровка фразы Stirlitz - You are a Fool!!

Теперь Центр получит сообщение Stirlitz - You are a Hero!!, а Мюллер - Stirlitz - You are a Fool!!.

3 Ответы на контрольные вопросы

1. Поясните смысл однократного гаммирования.

- Однократное гаммирование - это наложение (снятие) на открытые (зашифрованные) данные последовательности элементов других данных, которое использует однократную вероятностную гамму той же длины, что и подлежащий сокрытию текст.

2. Перечислите недостатки однократного гаммирования.

- Необходимость в полной случайности ключа TRNG;
- Если злоумышленник знает часть исходного текста и соответствующую шифрограмму, то текст можно расшифровать;
- Использование одной и той же гаммы на разные тексты снижает уровень защиты.

3. Перечислите преимущества однократного гаммирования.

- Даже при раскрытии части последовательности гаммы нельзя получить информацию о всём скрываемом тексте;
- Криптоалгоритм не даёт никакой информации об открытом тексте;
- Вычислительная эффективность.

4. Почему длина открытого текста должна совпадать с длиной ключа?

- В противном случае, нарушается абсолютная стойкость шифротекста. Шифр становится уязвим для частотного анализа и других атак.

5. Какая операция используется в режиме однократного гаммирования, назовите её особенности?
- Применяется операция XOR (исключающее ИЛИ, сложение по модулю 2), её особенность в том, что двойное прибавление одной и той же величины восстанавливает исходное значение, также шифрование и расшифрование выполняется одной и той же программой.
6. Как по открытому тексту и ключу получить шифротекст?
- Применить операцию XOR между последовательностями символов текста и ключа.
7. Как по открытому тексту и шифротексту получить ключ?
- Применить операцию XOR между последовательностями символов текста и шифротекста.
8. В чем заключаются необходимые и достаточные условия абсолютной стойкости шифра?
- Полная случайность ключа;
 - Равенство длин ключа и открытого текста;
 - Однократное использование ключа.

4 Выводы

В результате выполнения лабораторной работы я самостоятельно написал программу, выполняющую однократное гаммирование на заданных примерах. Она способна шифровать сообщение по ключу и открытому тексту, дешифровать по ключу и шифротексту, создавать новый ключ на основе шифротекста и открытого текста. Я на практике освоил одну из простых, но надежных схем шифрования данных.

Список литературы

1. Кулябов Д. С., Королькова А. В., Геворкян М. Н. Основы информационной безопасности (09.03.03) : Лабораторная работа №7. — URL: https://esystem.rudn.ru/pluginfile.php/3096814/mod_resource/content/2/007-lab_crypto-gamma.pdf (дата обр. 16.05.2026).